

In Java, a **Function** is a **functional interface** introduced in **Java 8** in the `java.util.function` package. It represents an operation that **takes one input (argument) and produces one output (result)**.

Unlike a Predicate, which returns only true or false, a Function can return **any type of value**.

Definition

The Function interface is defined as:

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

Where:

- **T** = Type of the input (argument)
 - **R** = Type of the output (return value)
-

Syntax

`Function<T, R> function = input -> output;`

Example:

```
Function<Integer, String> convert = number -> "Number: " + number;
```

```
System.out.println(convert.apply(100));
```

Output:

Number: 100

Here:

- Input (T) = Integer
 - Output (R) = String
-

Real-Life Analogy

Imagine a **fruit juice machine**.

Apple —————▶ Juice Machine —————▶ Apple Juice

- Input = Apple
- Processing = Juice Machine
- Output = Apple Juice

Similarly, a Java Function:

Input (T) —————▶ Function —————▶ Output (R)

Method in Function

The only abstract method is:

R apply(T t);

It takes one input and returns one result.

Example 1: Square a Number

```
import java.util.function.Function;

public class Demo {

    public static void main(String[] args) {

        Function<Integer, Integer> square = n -> n * n;

        System.out.println(square.apply(5));
        System.out.println(square.apply(10));
    }
}
```

Output:

25
100

Example 2: Convert String to Uppercase

```
Function<String, String> upperCase =
    str -> str.toUpperCase();

System.out.println(upperCase.apply("java"));
```

Output:

JAVA

Example 3: Find String Length

```
Function<String, Integer> length =  
    str -> str.length();  
  
System.out.println(length.apply("Programming"));
```

Output:

11

Example 4: Convert Employee to Salary

```
class Employee {  
  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
}  
  
Function<Employee, Double> getSalary =  
    emp -> emp.salary;  
  
Employee emp = new Employee("Anurag", 50000);  
  
System.out.println(getSalary.apply(emp));
```

Output:

50000.0

Understanding <T, R>

Suppose we write:

```
Function<String, Integer>
```

It means:

Input : String
Output : Integer

Example:

```
Function<String, Integer> length =  
    s -> s.length();  
"Java"  
  |  
  ▼  
length()  
  |  
  ▼  
4
```

Another example:

```
Function<Integer, Double>  
Input : Integer  
Output : Double  
Function<Integer, Double> half =  
    n -> n / 2.0;
```

```
System.out.println(half.apply(5));
```

Output:

2.5

Function vs Predicate

Feature	Function	Predicate
Package	java.util.function	java.util.function
Input	One	One
Output	Any type	Boolean
Abstract Method	apply()	test()
Purpose	Transform data	Test a condition

Example:

```
Function<String, Integer> length =
```

```
s -> s.length();
```

```
Predicate<String> isLong =
```

```
    s -> s.length() > 5;
```

```
Input : "Programming"
```

```
Function
```

```
↓
```

```
11
```

```
Predicate
```

```
↓
```

```
true
```

Function vs Consumer vs Supplier vs Predicate

Interface	Input	Output	Method
Function<T,R>	Yes	Yes	apply()
Predicate<T>	Yes	boolean	test()
Consumer<T>	Yes	No	accept()
Supplier<T>	No	Yes	get()

Built-in Default Methods

The Function interface provides useful default methods.

1. andThen()

Applies another function **after** the current one.

```
Function<Integer, Integer> multiply = n -> n * 2;
```

```
Function<Integer, Integer> add = n -> n + 5;
```

```
System.out.println(multiply.andThen(add).apply(10));
```

Execution:

```
10
```

```
↓
```

```
×2
```

```
↓
```

20
↓
+5
↓
25

Output:

25

2. compose()

Applies another function **before** the current one.

```
Function<Integer, Integer> multiply = n -> n * 2;
```

```
Function<Integer, Integer> add = n -> n + 5;
```

```
System.out.println(multiply.compose(add).apply(10));
```

Execution:

10
↓
+5
↓
15
↓
×2
↓
30

Output:

30

3. identity()

Returns the same input unchanged.

```
Function<String, String> identity = Function.identity();
```

```
System.out.println(identity.apply("Java"));
```

Output:

Java

Real-Time Example with Stream API

Suppose we have a list of names and want to convert them to uppercase.

```
import java.util.Arrays;
import java.util.List;
import java.util.function.Function;

public class Demo {

    public static void main(String[] args) {

        List<String> names =
            Arrays.asList("java", "python", "c++");

        Function<String, String> upper =
            String::toUpperCase;

        names.stream()
            .map(upper)
            .forEach(System.out::println);
    }
}
```

Output:

```
JAVA
PYTHON
C++
```

Here, `map()` expects a `Function<? super T, ? extends R>`, and `upper` transforms each `String` into another `String`.

Summary

Concept	Description
Function<T, R>	A functional interface that transforms one value into another.
T	Input type (argument).
R	Output type (return value).
apply(T t)	Performs the transformation.

andThen()	Chains another function after the current one.
compose()	Applies another function before the current one.
identity()	Returns the input unchanged.
Common Use Cases	Stream map(), data transformation, object conversion, DTO mapping, formatting, parsing, calculations.

A good way to remember the four most common functional interfaces is:

- **Function** → *Input* → *Output* (transforms data)
- **Predicate** → *Input* → *boolean* (tests a condition)
- **Consumer** → *Input* → *no output* (performs an action)
- **Supplier** → *No input* → *Output* (provides a value)

Yes. **Function**, **Predicate**, **Consumer**, and **Supplier** are all **functional interfaces** in Java 8. They are part of the `java.util.function` package and were introduced to support **Lambda Expressions**, **Method References**, and the **Stream API**.

What is a Functional Interface?

A **Functional Interface** is an interface that contains **exactly one abstract method** (**SAM - Single Abstract Method**).

It may also contain:

- Default methods (default)
- Static methods (static)
- Methods inherited from Object (`toString()`, `equals()`, `hashCode()`)

Since there is only one abstract method, Java knows which method a lambda expression should implement.

Example:

```
@FunctionalInterface
interface Greeting {

    void sayHello();

}
```

Lambda Expression:

```
Greeting g = () -> System.out.println("Hello");
```

```
g.sayHello();
```

Output

Hello

Why did Java introduce these interfaces?

Before Java 8:

```
Collections.sort(list, new Comparator<String>() {
```

```
    @Override
    public int compare(String a, String b) {
        return a.compareTo(b);
    }
});
```

After Java 8:

```
Collections.sort(list, (a, b) -> a.compareTo(b));
```

The lambda works because Comparator is a functional interface.

Similarly,

- Function
- Predicate
- Consumer
- Supplier

allow passing behavior as data.

1. Function<T, R>

Definition

Represents a function that accepts one argument and returns one result.

Input → Output

Generic declaration

Function<T, R>

- T = Input type
- R = Return type

Abstract Method

R apply(T t);

Example

Convert String into Integer

```
import java.util.function.Function;

public class Demo {

    public static void main(String[] args) {

        Function<String, Integer> length =
            str -> str.length();

        System.out.println(length.apply("Java"));

    }
}
```

Output

4

Explanation

"Java"
|
▼
Function
|
▼
4

Another Example

Square of a number

```
Function<Integer, Integer> square =  
    n -> n * n;
```

```
System.out.println(square.apply(5));
```

Output

25

Real-world Example

Employee Salary

```
Function<Employee, Double> salary =  
    emp -> emp.getSalary();
```

Input

Employee

Output

Salary

Built-in Methods of Function

Method	Purpose
apply()	Executes the function
compose()	Executes another function first
andThen()	Executes another function after
identity()	Returns input unchanged

Example

```
Function<Integer,Integer> square = x -> x*x;  
Function<Integer,Integer> add10 = x -> x+10;
```

```
System.out.println(square.andThen(add10).apply(5));
```

Output

35

Calculation

5
↓
square
25
↓
add10
35

2. Predicate

Definition

Represents a function that accepts one argument and returns a **boolean**.

Input → true/false

Generic declaration

Predicate<T>

Abstract Method

boolean test(T t);

Example

```
Predicate<Integer> even =  
    n -> n % 2 == 0;
```

```
System.out.println(even.test(8));  
System.out.println(even.test(5));
```

Output

```
true  
false
```

Explanation

8
|
▼

Predicate



true

Another Example

Age verification

```
Predicate<Integer> adult =  
    age -> age >= 18;
```

```
System.out.println(adult.test(21));
```

Output

true

Stream Example

```
List<Integer> list =  
Arrays.asList(1,2,3,4,5,6);
```

```
list.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Output

2
4
6

Here,

filter() internally uses a Predicate.

Built-in Methods

Method	Purpose
test()	Executes predicate
and()	Logical AND

or()	Logical OR
negate()	Logical NOT
isEqual()	Equality check

Example

```
Predicate<Integer> positive = x -> x > 0;
Predicate<Integer> even = x -> x % 2 == 0;
```

```
System.out.println(
    positive.and(even).test(20)
);
```

Output

true

3. Consumer

Definition

Represents an operation that accepts one argument but returns **nothing**.

Input → No Output

Generic declaration

```
Consumer<T>
```

Abstract Method

```
void accept(T t);
```

Example

```
Consumer<String> print =
    s -> System.out.println(s);

print.accept("Hello");
```

Output

Hello

Explanation

Hello



Consumer



Print

No value is returned.

Another Example

```
Consumer<Integer> cube =  
    x -> System.out.println(x*x*x);
```

```
cube.accept(3);
```

Output

27

Stream Example

```
List<String> names =  
Arrays.asList("Ram","Shyam","Mohan");
```

```
names.forEach(System.out::println);
```

Output

```
Ram  
Shyam  
Mohan
```

forEach() expects a Consumer.

Built-in Methods

Method	Purpose
accept()	Executes consumer
andThen()	Executes next consumer

Example

```
Consumer<String> upper =
s -> System.out.println(s.toUpperCase());
```

```
Consumer<String> lower =
s -> System.out.println(s.toLowerCase());
```

```
upper.andThen(lower).accept("Java");
```

Output

```
JAVA
java
```

4. Supplier

Definition

Represents an operation that **does not take any input** but returns a value.

No Input → Output

Generic declaration

```
Supplier<T>
```

Abstract Method

```
T get();
```

Example

```
Supplier<Double> random =
() -> Math.random();
```

```
System.out.println(random.get());
```

Output

0.834572

Another Example

```
Supplier<String> company =  
    () -> "OpenAI";  
  
System.out.println(company.get());
```

Output

OpenAI

Stream Example

```
Supplier<List<String>> supplier =  
    ArrayList::new;  
  
List<String> list = supplier.get();
```

Built-in Methods

Method	Purpose
get()	Returns value

Comparison of the Four Interfaces

Interface	Input	Output	Main Method	Typical Use
Function<T,R>	Yes (1)	Yes	apply()	Transform data
Predicate<T>	Yes (1)	boolean	test()	Filter or validate
Consumer<T>	Yes (1)	No (void)	accept()	Perform side effects (print, save, log)
Supplier<T>	No	Yes	get()	Generate or provide values

Simple Analogy

Imagine an employee management system:

Interface	Real-life Analogy	Example
Function	Convert employee to salary	Employee → ₹50,000
Predicate	Check if employee is eligible	Employee → true
Consumer	Print employee details	Employee → Display on screen
Supplier	Generate a new employee ID	No input → "EMP1001"

When to Use Each Interface

Situation	Use
Transform one object into another	Function
Test a condition	Predicate
Perform an action without returning a value	Consumer
Create or provide a value without input	Supplier

Summary

Interface	Is Functional Interface?	Abstract Method	Input	Output
Function<T,R>	✓ Yes	apply(T)	1	1
Predicate<T>	✓ Yes	test(T)	1	Boolean
Consumer<T>	✓ Yes	accept(T)	1	Void
Supplier<T>	✓ Yes	get()	0	1

These four interfaces are the most commonly used functional interfaces in Java 8. They underpin many APIs, especially the Stream API (map() uses Function, filter() uses Predicate, forEach() uses Consumer) and lazy value creation with Supplier.

Predicate<? super T> predicate is a Java Generics declaration that is commonly used in the Java Stream API and Collection Framework. Understanding it requires breaking it into smaller parts.

1. What is Predicate?

A **Predicate** is a **functional interface** in Java (java.util.function.Predicate<T>) that represents a function which:

- Accepts one input.
- Returns either true or false.

Definition

```
public interface Predicate<T> {  
    boolean test(T t);  
}
```

- Input → One object of type T
 - Output → boolean
-

Example

```
Predicate<Integer> isEven = n -> n % 2 == 0;
```

```
System.out.println(isEven.test(10));  
System.out.println(isEven.test(15));
```

Output

```
true  
false
```

Here

Integer → Input

boolean → Output

2. What is T?

T is a **Type Parameter** (Generic Type).

It represents any datatype.

Example

```
Predicate<Integer>  
Predicate<String>  
Predicate<Employee>  
Predicate<Student>
```

Here

```
T = Integer  
T = String  
T = Employee
```

3. What is ?

The question mark (?) is called a **Wildcard**.

It means

"I don't know the exact type."

Example

```
List<?> list;
```

It means

```
List<Integer>  
List<String>  
List<Employee>
```

Any list.

4. What is super?

super means

Any superclass of a given type.

Syntax

? super T

means

T
or
Parent of T
or
Grandparent of T
or
Object

Example

Class hierarchy

Object
↑
Animal
↑
Dog

If

T = Dog

then

? super Dog

can be

Dog
Animal
Object

NOT

Puppy

because Puppy is a child.

5. Meaning of Predicate<? super T>

Suppose

T = Dog

Then

Predicate<? super Dog>

means

The Predicate can accept

Predicate<Dog>

or

Predicate<Animal>

or

Predicate<Object>

Example

```
class Animal {  
}
```

```
class Dog extends Animal {  
}
```

Now

```
Predicate<Animal> p = animal -> true;
```

```
Predicate<? super Dog> predicate = p;
```

Valid

because

Animal

is a superclass of

Dog

Example 2

```
Predicate<Object> p = obj -> true;
```

```
Predicate<? super String> predicate = p;
```

Valid

because

Object

is superclass of

String

Example 3

```
Predicate<String> p = s -> s.length() > 5;
```

```
Predicate<? super String> predicate = p;
```

Also valid.

Invalid Example

```
Predicate<Dog> p = dog -> true;
```

```
Predicate<? super Animal> predicate = p;
```

Not allowed.

Reason

Dog

is NOT a superclass of

Animal

It is the child.

Why use ? super T?

Imagine Java used

```
Predicate<T>
```

only.

Then only

Predicate<Dog>

would be accepted.

But Java wants more flexibility.

Suppose

Predicate<Object> p = obj -> true;

Obviously

Dog

is an Object.

So Java allows it.

Real Example from Stream API

Method signature

Stream<T> filter(Predicate<? super T> predicate)

Suppose

Stream<String>

Then

T = String

The method accepts

Predicate<String>

Predicate<Object>

Predicate<CharSequence>

because

Object

↑

CharSequence

↑

String

Example

```
Predicate<Object> p = obj -> obj != null;
```

```
Stream<String> stream =  
    Stream.of("A", "B", null);
```

```
stream.filter(p)  
    .forEach(System.out::println);
```

Output

```
A  
B
```

Predicate<Object> works because every String is an Object.

Another Example

```
Predicate<CharSequence> p =  
    s -> s.length() > 2;
```

```
Stream<String> stream =  
    Stream.of("Java", "C", "Python");
```

```
stream.filter(p)  
    .forEach(System.out::println);
```

Output

```
Java  
Python
```

CharSequence is the parent of String.

PECS Rule

Java Generics follow the famous rule:

PECS = Producer Extends, Consumer Super

Producer → extends

If data is **produced (read)**

<? extends T>

Consumer → super

If data is **consumed (accepted as input)**

<? super T>

A Predicate **consumes** objects by testing them, so super is appropriate.

Type Hierarchy Example

Object
↑
CharSequence
↑
String

For

T = String

| Type | Allowed for ? super String |
|-----------------------------|----------------------------|
| Predicate | ✓ Yes |
| Predicate | ✓ Yes |
| Predicate | ✓ Yes |
| Predicate | ✗ No |
| Predicate | ✗ No |
| Predicate<? extends String> | ✗ Not equivalent |

Visual Representation

Object
↑

Animal

↑

Dog

If

Predicate<? super Dog>

Allowed

Predicate<Dog>

Predicate<Animal>

Predicate<Object>

Not Allowed

Predicate<Puppy>

because

Puppy

is below Dog.

Summary

| Term | Meaning |
|-----------------------------------|--|
| Predicate<T> | A functional interface that accepts a T and returns boolean. |
| T | A generic type parameter representing any type. |
| ? | Wildcard representing an unknown type. |
| ? super T | T or any superclass of T. |
| Predicate<? super T> | A predicate that can test objects of type T using logic defined for T or any of its supertypes. |
| Why super? | Because Predicate consumes values of type T, following the PECS (Producer Extends, Consumer Super) guideline. |

Java 8 introduced the **java.util.function** package, which contains built-in **functional interfaces** used with Lambda Expressions, the Stream API, and Method References.

1. Core Functional Interfaces

| Functional Interface | Abstract Method | Input | Output | Description | Example |
|--------------------------------|-----------------|-------|-----------|--------------------------------|------------------|
| Function<T,R> | apply(T) | 1 | 1 | Converts one object to another | String → Integer |
| BiFunction<T,U,R> | apply(T,U) | 2 | 1 | Accepts two inputs | Add two numbers |
| UnaryOperator<T> | apply(T) | 1 | Same Type | Operates on same type | Square number |
| BinaryOperator<T> | apply(T,T) | 2 | Same Type | Combines two objects | Find maximum |

2. Predicate Interfaces

| Functional Interface | Abstract Method | Return Type | Purpose | Example |
|-------------------------------|-----------------|-------------|-------------------|-----------------|
| Predicate<T> | test(T) | boolean | Tests a condition | Age > 18 |
| BiPredicate<T,U> | test(T,U) | boolean | Tests two values | Compare strings |

3. Consumer Interfaces

| Functional Interface | Abstract Method | Return Type | Purpose | Example |
|------------------------------|-----------------|-------------|---------------------|-----------------|
| Consumer<T> | accept(T) | void | Consumes one value | Print value |
| BiConsumer<T,U> | accept(T,U) | void | Consumes two values | Print key-value |

4. Supplier Interfaces

| Functional Interface | Abstract Method | Return Type | Purpose | Example |
|--------------------------|-----------------|-------------|------------------|------------------------|
| Supplier<T> | get() | T | Supplies a value | Generate random number |
| BooleanSupplier | getAsBoolean() | boolean | Supplies boolean | Random true/false |
| IntSupplier | getAsInt() | int | Supplies int | Random int |
| LongSupplier | getAsLong() | long | Supplies long | Timestamp |
| DoubleSupplier | getAsDouble() | double | Supplies double | Random double |

5. Primitive Function Interfaces

| Interface | Method | Description |
|----------------------------------|---------------------|-----------------|
| IntFunction<R> | apply(int) | int → Object |
| LongFunction<R> | apply(long) | long → Object |
| DoubleFunction<R> | apply(double) | double → Object |
| ToIntFunction<T> | applyAsInt(T) | Object → int |
| ToLongFunction<T> | applyAsLong(T) | Object → long |
| ToDoubleFunction<T> | applyAsDouble(T) | Object → double |
| IntToLongFunction | applyAsLong(int) | int → long |
| IntToDoubleFunction | applyAsDouble(int) | int → double |
| LongToIntFunction | applyAsInt(long) | long → int |
| LongToDoubleFunction | applyAsDouble(long) | long → double |
| DoubleToIntFunction | applyAsInt(double) | double → int |
| DoubleToLongFunction | applyAsLong(double) | double → long |

6. Primitive Unary Operators

| Interface | Method | Description |
|----------------------------|-----------------|-----------------|
| IntUnaryOperator | applyAsInt() | int → int |
| LongUnaryOperator | applyAsLong() | long → long |
| DoubleUnaryOperator | applyAsDouble() | double → double |

7. Primitive Binary Operators

| Interface | Method | Description |
|-----------------------------|-----------------|--------------------------|
| IntBinaryOperator | applyAsInt() | int + int → int |
| LongBinaryOperator | applyAsLong() | long + long → long |
| DoubleBinaryOperator | applyAsDouble() | double + double → double |

8. Primitive Predicates

| Interface | Method | Description |
|------------------------|--------------|--------------|
| IntPredicate | test(int) | Tests int |
| LongPredicate | test(long) | Tests long |
| DoublePredicate | test(double) | Tests double |

9. Primitive Consumers

| Interface | Method | Description |
|-----------------------------------|------------------|-----------------|
| IntConsumer | accept(int) | Consumes int |
| LongConsumer | accept(long) | Consumes long |
| DoubleConsumer | accept(double) | Consumes double |
| ObjIntConsumer<T> | accept(T,int) | Object + int |
| ObjLongConsumer<T> | accept(T,long) | Object + long |
| ObjDoubleConsumer<T> | accept(T,double) | Object + double |

10. Primitive BiFunctions

| Interface | Method | Description |
|--------------------------------------|-----------------|----------------------|
| ToIntBiFunction<T,U> | applyAsInt() | Two objects → int |
| ToLongBiFunction<T,U> | applyAsLong() | Two objects → long |
| ToDoubleBiFunction<T,U> | applyAsDouble() | Two objects → double |

11. Primitive Function Variants

| Interface | Description |
|--------------------------------|------------------|
| IntConsumer | int → void |
| LongConsumer | long → void |
| DoubleConsumer | double → void |
| IntFunction<R> | int → Object |
| LongFunction<R> | long → Object |
| DoubleFunction<R> | double → Object |
| IntPredicate | int → boolean |
| LongPredicate | long → boolean |
| DoublePredicate | double → boolean |
| IntSupplier | () → int |
| LongSupplier | () → long |
| DoubleSupplier | () → double |

Default (Built-in) Methods in Functional Interfaces

| Interface | Built-in Methods |
|-----------------|---|
| Function | apply(), compose(), andThen(), identity() |

| | |
|-----------------------|--|
| Predicate | test(), and(), or(), negate(), isEqual() |
| Consumer | accept(), andThen() |
| Supplier | get() |
| UnaryOperator | apply(), identity() |
| BinaryOperator | apply(), minBy(), maxBy() |
| BiFunction | apply(), andThen() |
| BiPredicate | test(), and(), or(), negate() |
| BiConsumer | accept(), andThen() |

Total Functional Interfaces in Java 8 (java.util.function)

| Category | Count |
|---|-----------|
| Core (Function, Predicate, Consumer, Supplier, etc.) | 9 |
| Primitive Specializations | 34 |
| Primitive/Object Combination Interfaces | 10 |
| Total Functional Interfaces | 43 |

These interfaces form the foundation for Java 8 features such as **Lambda Expressions**, **Method References**, **Stream API**, **Optional**, and the **Collections API**.

The **Stream API** and **Lambda Expressions** in Java 8 rely heavily on the built-in **functional interfaces** provided in the `java.util.function` package. Stream operations such as `filter()`, `map()`, `flatMap()`, `reduce()`, `collect()`, `forEach()`, and `sorted()` expect one or more of these interfaces as parameters.

Important distinction

- **Functional interfaces** are interfaces with a single abstract method (e.g., `Function`, `Predicate`, `Consumer`).
- **Functions** (such as lambda expressions or method references) are implementations of those functional interfaces.

For example:

```
Function<String, Integer> length = s -> s.length();
```

Here:

- `Function<String, Integer>` is the **functional interface**.
- `s -> s.length()` is the **function (lambda expression)**.

Complete List of Functional Interfaces Commonly Used with Stream API and Lambda Expressions

| Functional Interface | Abstract Method | Input | Output | Common Stream Methods |
|--------------------------------|--------------------------|-------|-----------|---|
| Predicate<T> | <code>test(T)</code> | 1 | boolean | <code>filter()</code> , <code>removeIf()</code> |
| BiPredicate<T,U> | <code>test(T,U)</code> | 2 | boolean | Custom logic |
| Function<T,R> | <code>apply(T)</code> | 1 | 1 | <code>map()</code> , <code>collect()</code> |
| BiFunction<T,U,R> | <code>apply(T,U)</code> | 2 | 1 | <code>reduce()</code> , <code>Map.compute()</code> |
| UnaryOperator<T> | <code>apply(T)</code> | 1 | Same type | <code>replaceAll()</code> |
| BinaryOperator<T> | <code>apply(T,T)</code> | 2 | Same type | <code>reduce()</code> , <code>maxBy()</code> , <code>minBy()</code> |
| Consumer<T> | <code>accept(T)</code> | 1 | void | <code>forEach()</code> , <code>peek()</code> |
| BiConsumer<T,U> | <code>accept(T,U)</code> | 2 | void | <code>Map.forEach()</code> , <code>collect()</code> |
| Supplier<T> | <code>get()</code> | 0 | 1 | <code>Stream.generate()</code> , <code>collect()</code> |

1. Predicate

Definition

Represents a condition that returns either true or false.

```
boolean test(T t);
```

Used By

- filter()
- removeIf()
- takeWhile() (Java 9)

Example

```
List<Integer> numbers =  
Arrays.asList(10,15,20,25,30);  
  
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Output

```
10  
20  
30
```

Flow

```
10 → true  
15 → false  
20 → true  
25 → false  
30 → true
```

2. Function<T,R>

Definition

Transforms one object into another.

```
R apply(T t);
```

Used By

- `map()`
- `Collectors.mapping()`

Example

```
List<String> names =  
Arrays.asList("Java","Python","C");  
  
names.stream()  
    .map(String::length)  
    .forEach(System.out::println);
```

Output

4
6
1

Flow

```
Java  
↓  
Function  
↓  
4
```

3. Consumer

Definition

Consumes data without returning anything.

```
void accept(T t);
```

Used By

- `forEach()`
- `peek()`

Example

```
Stream.of("A","B","C")  
    .forEach(System.out::println);
```

Output

A

B
C

4. Supplier

Definition

Produces values without taking any input.

`T get();`

Used By

- `Stream.generate()`
- `Collector`

Example

```
Stream.generate(() -> "Java")
    .limit(3)
    .forEach(System.out::println);
```

Output

Java
Java
Java

5. UnaryOperator

Definition

A special type of Function where the input and output types are the same.

`T apply(T t);`

Example

```
UnaryOperator<Integer> square =
    x -> x*x;

System.out.println(square.apply(5));
```

Output

25

6. BinaryOperator

Definition

Combines two values of the same type into one value of the same type.

```
T apply(T t1, T t2);
```

Used By

- `reduce()`

Example

```
List<Integer> list =  
Arrays.asList(1,2,3,4);  
  
int sum =  
list.stream()  
    .reduce(0, Integer::sum);
```

```
System.out.println(sum);
```

Output

10

7. BiFunction<T,U,R>

Definition

Accepts two inputs and returns one output.

```
R apply(T t,U u);
```

Example

```
BiFunction<Integer,Integer,Integer> add =  
(a,b)->a+b;
```

```
System.out.println(add.apply(5,10));
```

Output

15

8. BiConsumer<T,U>

Definition

Consumes two values.

```
void accept(T t,U u);
```

Example

```
Map<String,Integer> map =  
Map.of("A",10,"B",20);  
  
map.forEach(  
(name,marks)->  
System.out.println(name+" "+marks)  
);
```

Output

```
A 10  
B 20
```

9. BiPredicate<T,U>

Definition

Tests two values.

```
boolean test(T t,U u);
```

Example

```
BiPredicate<String,String> equal =  
(a,b)->a.equals(b);  
  
System.out.println(equal.test("Java","Java"));
```

Output

```
true
```

Primitive Functional Interfaces

Java provides primitive specializations to avoid boxing and unboxing overhead.

| Interface | Input | Output | Example |
|----------------------------------|---------------|---------|---------------------------|
| IntPredicate | int | boolean | x -> x > 0 |
| LongPredicate | long | boolean | x -> x > 100 |
| DoublePredicate | double | boolean | x -> x > 5.5 |
| IntFunction<R> | int | Object | x -> "No: "+x |
| LongFunction<R> | long | Object | ID lookup |
| DoubleFunction<R> | double | Object | Format value |
| ToIntFunction<T> | Object | int | String::length |
| ToLongFunction<T> | Object | long | Extract timestamp |
| ToDoubleFunction<T> | Object | double | Extract salary |
| IntConsumer | int | void | Print integer |
| LongConsumer | long | void | Log value |
| DoubleConsumer | double | void | Print decimal |
| IntSupplier | None | int | Random integer |
| LongSupplier | None | long | Current time |
| DoubleSupplier | None | double | Random double |
| IntUnaryOperator | int | int | Square |
| LongUnaryOperator | long | long | Double value |
| DoubleUnaryOperator | double | double | Square root approximation |
| IntBinaryOperator | int,int | int | Sum |
| LongBinaryOperator | long,long | long | Max |
| DoubleBinaryOperator | double,double | double | Average |

Functional Interfaces Used by Major Stream Operations

| Stream Method | Functional Interface Required | Example |
|--|----------------------------------|--------------------------------|
| filter() | Predicate<T> | n -> n > 10 |
| map() | Function<T,R> | String::length |
| flatMap() | Function<T, Stream<R>> | list -> list.stream() |
| peek() | Consumer<T> | System.out::println |
| forEach() | Consumer<T> | System.out::println |
| forEachOrdered() | Consumer<T> | Print in encounter order |
| reduce(BinaryOperator) | BinaryOperator<T> | Integer::sum |
| reduce(identity, accumulator) | BinaryOperator<T> | (a,b) -> a + b |
| reduce(identity, accumulator, combiner) | BiFunction + BinaryOperator | Parallel reduction |
| collect() | Supplier, BiConsumer, BiConsumer | Collectors.toList() internally |
| sorted() | Comparator<T>* | Comparator.naturalOrder() |
| min() | Comparator<T>* | Compare elements |
| max() | Comparator<T>* | Compare elements |
| anyMatch() | Predicate<T> | x -> x > 0 |
| allMatch() | Predicate<T> | x -> x > 0 |
| noneMatch() | Predicate<T> | x -> x < 0 |

Note: Comparator<T> is also a functional interface (from java.util) even though it is **not** in the java.util.function package. It is widely used with lambdas in sorted(), min(), and max().

Other Functional Interfaces Frequently Used with Lambdas (Outside java.util.function)

| Interface | Package | Used In |
|----------------------------|----------------------|------------------------------|
| Comparator<T> | java.util | sorted(), Collections.sort() |
| Runnable | java.lang | Threads |
| Callable<V> | java.util.concurrent | Executors |
| ActionListener | java.awt.event | GUI event handling |
| FileFilter | java.io | File operations |
| FilenameFilter | java.io | File name filtering |

Summary

| Functional Interface | Purpose | Returns |
|-----------------------|-------------------------|-----------|
| Predicate | Test a condition | boolean |
| Function | Transform data | Any type |
| Consumer | Consume data | void |
| Supplier | Produce data | Any type |
| UnaryOperator | Transform same type | Same type |
| BinaryOperator | Combine same types | Same type |
| BiFunction | Two inputs → one output | Any type |
| BiPredicate | Test two inputs | Boolean |
| BiConsumer | Consume two inputs | Void |
| Comparator | Compare two values | Int |

These interfaces are the foundation of Java's functional programming model. Almost every lambda expression used with the Stream API implements one of these interfaces, either directly (such as Predicate, Function, or Consumer) or indirectly through specialized interfaces like Comparator or primitive specializations such as IntPredicate and ToIntFunction.

Method References in Java 8

A **Method Reference** is a shorthand syntax for a lambda expression that **calls an existing method**. Instead of writing a lambda that simply invokes a method, you can use the `::` operator.

For example:

```
// Lambda
Consumer<String> printer = s -> System.out.println(s);
```

```
// Method Reference
Consumer<String> printer = System.out::println;
```

Both are equivalent.

Types of Method References

Java supports four types of method references.

| Type | Syntax | Example |
|--|---------------------------|-------------------|
| Static Method | ClassName::staticMethod | Integer::parseInt |
| Instance Method of a Particular Object | object::instanceMethod | scanner::nextLine |
| Instance Method of an Arbitrary Object of a Particular Type | ClassName::instanceMethod | String::length |
| Constructor Reference | ClassName::new | ArrayList::new |

Functional Interfaces Commonly Used with Method References

1. Function<T,R>

Definition

Represents a function that accepts one input and returns one output.

```
R apply(T t);
```

Method Reference Example

```
import java.util.function.Function;

Function<String, Integer> length = String::length;

System.out.println(length.apply("Java"));
```

Equivalent Lambda

```
Function<String, Integer> length =
    s -> s.length();
```

Output

4

2. Predicate

Definition

Represents a condition.

```
boolean test(T t);
```

Method Reference Example

```
Predicate<String> empty =
    String::isEmpty;

System.out.println(empty.test(""));
System.out.println(empty.test("Java"));
```

Equivalent Lambda

```
Predicate<String> empty =
    s -> s.isEmpty();
```

Output

```
true
false
```

3. Consumer

Definition

Consumes one object and returns nothing.

```
void accept(T t);
```

Method Reference Example

```
Consumer<String> print =  
    System.out::println;
```

```
print.accept("Hello");
```

Equivalent Lambda

```
Consumer<String> print =  
    s -> System.out.println(s);
```

Output

Hello

4. Supplier

Definition

Produces a value without taking input.

```
T get();
```

Method Reference Example

```
Supplier<ArrayList<String>> supplier =  
    ArrayList::new;
```

```
ArrayList<String> list = supplier.get();
```

Equivalent Lambda

```
Supplier<ArrayList<String>> supplier =  
    () -> new ArrayList<>();
```

5. UnaryOperator

Definition

Accepts one value and returns the same type.

```
T apply(T t);
```

Method Reference Example

```
UnaryOperator<String> upper =  
    String::toUpperCase;  
  
System.out.println(upper.apply("java"));
```

Output

JAVA

Equivalent Lambda

```
UnaryOperator<String> upper =  
    s -> s.toUpperCase();
```

6. BinaryOperator

Definition

Accepts two values of the same type and returns the same type.

```
T apply(T a, T b);
```

Method Reference Example

```
BinaryOperator<Integer> sum =  
    Integer::sum;  
  
System.out.println(sum.apply(10,20));
```

Equivalent Lambda

```
BinaryOperator<Integer> sum =  
    (a,b) -> a+b;
```

Output

30

7. BiFunction<T,U,R>

Definition

Accepts two inputs and returns one output.

```
R apply(T t,U u);
```

Method Reference Example

```
BiFunction<String,String,String> concat =  
    String::concat;
```

```
System.out.println(  
    concat.apply("Java","8")  
);
```

Equivalent Lambda

```
(a,b)->a.concat(b)
```

Output

Java8

8. BiPredicate<T,U>

Definition

Tests two values.

```
boolean test(T t,U u);
```

Method Reference Example

```
BiPredicate<String,String> equal =  
    String::equals;
```

```
System.out.println(equal.test("Java","Java"));
```

Equivalent Lambda

```
(a,b)->a.equals(b)
```

Output

true

9. BiConsumer<T,U>

Definition

Consumes two values.

```
void accept(T t,U u);
```

Method Reference Example

```
Map<String,Integer> map =  
Map.of("A",10,"B",20);  
  
map.forEach(System.out::println);
```

Equivalent Lambda

```
map.forEach((k,v)->  
System.out.println(k+" "+v));
```

Output

```
A 10  
B 20
```

Primitive Functional Interfaces

IntPredicate

```
IntPredicate positive =  
    x -> x>0;
```

Method reference

```
IntPredicate positive =  
    MyClass::isPositive;
```

IntConsumer

```
IntConsumer printer =  
    System.out::println;
```

IntSupplier

```
IntSupplier random =  
    randomGenerator::nextInt;
```

IntUnaryOperator

```
IntUnaryOperator abs =  
    Math::abs;
```

IntBinaryOperator

```
IntBinaryOperator max =  
    Math::max;
```

Comparator

Although not in `java.util.function`, it is one of the most frequently used functional interfaces with method references.

```
Comparator<String> cmp =  
    String::compareTo;
```

Equivalent Lambda

```
(a,b)->a.compareTo(b)
```

Example

```
List<String> list =  
Arrays.asList("B","A","C");  
  
list.sort(String::compareTo);
```

Output

```
A  
B  
C
```

Runnable

```
Runnable task =  
    Demo::print;
```

Equivalent Lambda

```
()->print();
```

Callable

```
Callable<String> task =  
    Demo::message;
```

Equivalent Lambda

```
()->message();
```

Stream API Examples

forEach()

```
list.forEach(System.out::println);
```

Uses

Consumer<T>

map()

```
names.stream()  
    .map(String::length)
```

Uses

Function<T,R>

filter()

```
stream.filter(String::isEmpty);
```

Uses

Predicate<T>

sorted()

```
list.sort(String::compareToIgnoreCase);
```

Uses

Comparator<T>

reduce()

```
numbers.stream()  
    .reduce(Integer::sum);
```

Uses

BinaryOperator<T>

collect()

```
stream.collect(Collectors.toList());
```

Internally uses

- Supplier
 - BiConsumer
 - BinaryOperator
-

Constructor References

Supplier

```
Supplier<List<String>> supplier =  
    ArrayList::new;
```

Equivalent Lambda

```
()->new ArrayList<>()
```

Function

```
Function<String,Integer> parser =  
    Integer::new
```

Equivalent Lambda

```
s -> new Integer(s)
```

Note: This constructor is deprecated in modern Java. Prefer `Integer::valueOf` or `Integer.parseInt` in new code.

Static Method References

Example

```
Function<String,Integer> parser =  
    Integer::parseInt;
```

Equivalent Lambda

```
s -> Integer.parseInt(s)
```

Instance Method References

Example

```
Function<String,String> upper =  
    String::toUpperCase;
```

Equivalent Lambda

```
s -> s.toUpperCase();
```

Instance Method of Particular Object

```
Scanner scanner =  
new Scanner(System.in);
```

```
Supplier<String> input =  
scanner::nextLine;
```

Equivalent Lambda

```
()->scanner.nextLine()
```

Complete Summary Table

| Functional Interface | Abstract Method | Common Method Reference Example | Lambda Equivalent |
|--------------------------------|-----------------|--|--|
| Function<T,R> | apply() | String::length | s -> s.length() |
| Predicate<T> | test() | String::isEmpty | s -> s.isEmpty() |
| Consumer<T> | accept() | System.out::println | x -> System.out.println(x) |
| Supplier<T> | get() | ArrayList::new | () -> new ArrayList<>() |
| UnaryOperator<T> | apply() | String::toUpperCase | s -> s.toUpperCase() |
| BinaryOperator<T> | apply() | Integer::sum | (a,b) -> a + b |
| BiFunction<T,U,R> | apply() | String::concat | (a,b) -> a.concat(b) |
| BiPredicate<T,U> | test() | String::equals | (a,b) -> a.equals(b) |
| BiConsumer<T,U> | accept() | System.out::println
(for Map.forEach) | (k,v) ->
System.out.println(k + " " + |

| | | | |
|----------------------------|--------------|---------------------|----------------------------|
| | | | v) |
| Comparator<T> | compare() | String::compareTo | (a,b) -> a.compareTo(b) |
| Runnable | run() | Demo::print | () -> print() |
| Callable<V> | call() | Demo::message | () -> message() |
| IntConsumer | accept() | System.out::println | x -> System.out.println(x) |
| IntPredicate | test() | MyClass::isPositive | x -> isPositive(x) |
| IntSupplier | getAsInt() | random::nextInt | () -> random.nextInt() |
| IntUnaryOperator | applyAsInt() | Math::abs | x -> Math.abs(x) |
| IntBinaryOperator | applyAsInt() | Math::max | (a,b) -> Math.max(a,b) |

Key Takeaways

- **Method references (::)** are simply a concise way to write lambda expressions that invoke an existing method or constructor.
- The most commonly used functional interfaces with method references are Function, Predicate, Consumer, Supplier, UnaryOperator, BinaryOperator, and Comparator.
- Method references can refer to **static methods, instance methods of a particular object, instance methods of an arbitrary object of a particular type, or constructors**.
- Every valid method reference must be compatible with the **single abstract method** of the target functional interface.

Java's Functional Programming Model

Java's **functional programming model** is a programming style introduced in **Java 8** that allows developers to write programs by treating **functions as first-class behavior**. Instead of focusing only on objects and mutable state, it focuses on **what should be done** rather than **how to do it**.

It enables developers to write **concise, readable, reusable, parallelizable, and maintainable** code using features such as:

- Lambda expressions (->)
 - Functional interfaces
 - Method references (::)
 - Stream API
 - Optional
 - Default and static methods in interfaces
 - Built-in functional interfaces (Function, Predicate, Consumer, Supplier, etc.)
-

Definition

Functional Programming (FP) is a programming paradigm in which computation is performed by evaluating functions instead of executing sequences of commands.

In Java, the functional programming model allows you to:

- Pass functions as arguments
 - Return functions from methods (through functional interfaces)
 - Store functions in variables
 - Compose functions together
 - Avoid unnecessary mutable state
 - Process collections declaratively
-

Before Java 8 (Imperative Programming)

Example

```
List<String> names = Arrays.asList("John", "Alex", "Bob");
```

```
for (String name : names) {  
    System.out.println(name);  
}
```

Characteristics

- Explicit loops

- Mutable variables
- Step-by-step instructions
- More boilerplate code

Java 8 Functional Style

```
List<String> names = Arrays.asList("John", "Alex", "Bob");
```

```
names.forEach(System.out::println);
```

Instead of saying

Loop through every element and print it.

You simply say

Print every element.

This is called **declarative programming**.

Imperative vs Functional

| Imperative Programming | Functional Programming |
|---------------------------|--------------------------|
| Focuses on how | Focuses on what |
| Uses loops | Uses Streams |
| Mutable variables | Prefer immutable data |
| Step-by-step instructions | Function composition |
| More code | Less code |
| Harder to parallelize | Easy parallel processing |

Building Blocks of Java Functional Programming

1. Functional Interfaces

A functional interface contains exactly one abstract method.

Example

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Lambda

```
Calculator c = (a, b) -> a + b;
```

```
System.out.println(c.add(5, 3));
```

Output

8

2. Lambda Expressions

A lambda expression is an anonymous function.

Syntax

(parameters) -> expression

Example

(a, b) -> a + b

Equivalent anonymous class

```
Calculator c = new Calculator() {
    @Override
    public int add(int a, int b) {
        return a + b;
    }
};
```

Lambda removes unnecessary boilerplate.

3. Method References

Instead of

```
names.forEach(name -> System.out.println(name));
```

Use

```
names.forEach(System.out::println);
```

Method references improve readability.

Types include:

- Static method reference
- Instance method reference
- Instance method of arbitrary object
- Constructor reference

Example

```
ArrayList<String> list = Stream.of("A", "B")  
    .collect(Collectors.toCollection(ArrayList::new));
```

4. Streams

A Stream processes data functionally.

Example

```
List<Integer> numbers =  
    Arrays.asList(5, 2, 8, 3, 6);  
  
numbers.stream()  
    .filter(n -> n % 2 == 0)  
    .map(n -> n * n)  
    .forEach(System.out::println);
```

Output

```
4  
36  
64
```

Processing steps:

```
5 2 8 3 6  
  |  
  filter()  
  |  
2 8 6  
  |  
  map()  
  |
```

```
4 64 36
  |
forEach()
```

5. Function Composition

Functions can be combined.

Example

```
Function<Integer, Integer> multiply = x -> x * 2;
Function<Integer, Integer> add = x -> x + 10;
```

```
Function<Integer, Integer> result =
    multiply.andThen(add);
```

```
System.out.println(result.apply(5));
```

Output

20

Explanation

5
↓

multiply by 2

10
↓

add 10

20

6. Predicate

Represents a condition.

```
Predicate<Integer> even =
    n -> n % 2 == 0;
```

```
System.out.println(even.test(8));
```

Output

true

Useful with Streams

```
numbers.stream()  
    .filter(even)  
    .forEach(System.out::println);
```

7. Function

Transforms one object into another.

```
Function<String, Integer> length =  
    String::length;  
  
System.out.println(length.apply("Java"));
```

Output

4

8. Consumer

Consumes data without returning anything.

```
Consumer<String> print =  
    System.out::println;  
  
print.accept("Hello");
```

Output

Hello

9. Supplier

Produces data without taking input.

```
Supplier<Double> random =  
    Math::random;  
  
System.out.println(random.get());
```

Example output

0.7345

10. Optional

Avoids NullPointerException.

Without Optional

```
String name = null;  
System.out.println(name.length());
```

Throws

NullPointerException

With Optional

```
Optional<String> name =  
    Optional.ofNullable(null);  
System.out.println(name.orElse("Unknown"));
```

Output

Unknown

Functional Programming Principles

1. Pure Functions

A pure function:

- Returns the same output for the same input.
- Has no side effects.

Example

```
int square(int x) {  
    return x * x;  
}
```

Calling

square(5)

always returns

25

2. Immutability

Avoid changing objects after creation.

Bad

```
list.add("Java");
```

Better

```
List<String> upper =  
    list.stream()  
        .map(String::toUpperCase)  
        .toList();
```

The original list remains unchanged.

3. Higher-Order Functions

A higher-order function accepts or returns another function.

Example

```
Function<Integer, Integer> square =  
    x -> x * x;
```

```
List<Integer> result =  
    numbers.stream()  
        .map(square)  
        .toList();
```

4. Declarative Programming

Imperative

```
for (Integer n : numbers) {  
    if (n > 5)  
        System.out.println(n);  
}
```

Declarative

```
numbers.stream()  
  .filter(n -> n > 5)  
  .forEach(System.out::println);
```

Real-World Example

Suppose you have employees.

```
class Employee {  
    String name;  
    double salary;  
  
    Employee(String name, double salary) {  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public double getSalary() {  
        return salary;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

List

```
List<Employee> employees = Arrays.asList(  
    new Employee("John", 50000),  
    new Employee("Alex", 70000),  
    new Employee("Bob", 90000)  
);
```

Find employees earning more than 60,000.

```
employees.stream()  
  .filter(e -> e.getSalary() > 60000)  
  .map(Employee::getName)  
  .forEach(System.out::println);
```

Output

Alex
Bob

Advantages

- Less boilerplate code
 - More readable and expressive
 - Easier to maintain
 - Encourages reusable functions
 - Better support for parallel processing (`parallelStream()`)
 - Reduced chance of bugs through immutability and pure functions
 - Better abstraction and modularity
-

Limitations

- Learning curve for developers used to imperative programming
 - Debugging long stream pipelines can be harder
 - Not every problem is naturally functional
 - Excessive chaining may reduce readability if overused
-

Functional Programming Flow

Collection



Stream



`filter()`



`map()`



`sorted()`



`distinct()`



`collect()`

Example

```
List<String> names = Arrays.asList(  
    "John", "Alex", "Bob", "John");
```

```
List<String> result = names.stream()
```

```
.filter(n -> n.length() > 3)
.map(String::toUpperCase)
.distinct()
.sorted()
.toList();
```

```
System.out.println(result);
```

Output

[ALEX, JOHN]

Summary

Java's functional programming model, introduced in Java 8, adds a declarative style of programming based on functions. It revolves around **functional interfaces, lambda expressions, method references, Stream API, Optional**, and **function composition**. By emphasizing pure functions, immutability, and higher-order functions, it enables developers to write cleaner, more expressive, reusable, and parallel-friendly code while still integrating seamlessly with Java's object-oriented programming model.

PECS Rule in Java Generics

PECS stands for:

- **P = Producer**
- **E = Extends**
- **C = Consumer**
- **S = Super**

The PECS rule helps you decide whether to use:

- `<? extends T>`
- `<? super T>`

It was popularized by Joshua Bloch in *Effective Java* and is one of the most important concepts in Java Generics.

Why do we need PECS?

Suppose we have the following classes:

```
class Animal { }
```

```
class Dog extends Animal { }
```

```
class Puppy extends Dog { }
```

Hierarchy:

```
Object
|
Animal
|
Dog
|
Puppy
```

Now consider:

```
List<Dog> dogs = new ArrayList<>();
```

Can we assign it to:

```
List<Animal> animals = dogs;
```

No.

Although Dog is an Animal, List<Dog> is **not** a subtype of List<Animal>.

This is why Java provides **wildcards**.

PECS Rule

| Situation | Wildcard |
|-----------------------|---------------|
| Read data (Producer) | <? extends T> |
| Write data (Consumer) | <? super T> |

A simple way to remember:

Producer → Extends

Consumer → Super

Producer = Extends

Definition

If a collection **produces objects for you to read**, use:

<? extends T>

Meaning:

"Any class that extends T."

Example

```
List<Dog> dogs = List.of(  
    new Dog(),  
    new Dog()  
);
```

```
List<? extends Animal> animals = dogs;
```

Here

Dog extends Animal

So this is valid.

Reading

```
Animal a = animals.get(0);
```

Works perfectly.

Why?

Because every Dog is an Animal.

Dog
↓
Animal

Writing

Can we do this?

```
animals.add(new Dog());
```

No.

Compile Error.

Why?

Because Java doesn't know the actual type.

It could actually be

```
List<Dog>
```

or

```
List<Puppy>
```

Imagine

```
List<Puppy> puppies = new ArrayList<>();
```

```
List<? extends Animal> animals = puppies;
```

Now

```
animals.add(new Dog());
```

would put a Dog inside a Puppy list.

Impossible.

Therefore Java prevents adding.

Allowed operations

```
Animal a = animals.get(0); // OK
```

```
animals.add(null); // OK
```

```
animals.add(new Dog()); // Error
```

Memory diagram

List<? extends Animal>

Actual List

Dog

Dog

Dog

↓

Read as Animal

Safe.

Consumer = Super

Definition

If a collection **consumes objects that you insert**, use

<? super T>

Meaning

Any superclass of T

Example

```
List<Animal> animals = new ArrayList<>();
```

```
List<? super Dog> list = animals;
```

Meaning

Animal

↑

Dog

Animal is a superclass of Dog.

Allowed.

Now

```
list.add(new Dog());
```

Works.

Also

```
list.add(new Puppy());
```

Works.

Because Puppy IS-A Dog.

But reading?

```
Dog d = list.get(0);
```

Compile Error.

Why?

Because Java only knows it returns

Object

The actual list may be

Animal

or

Object

So only this is safe:

```
Object obj = list.get(0);
```

Allowed operations

```
list.add(new Dog());
```

```
list.add(new Puppy());
```

```
Object o = list.get(0);
```

```
Dog d = list.get(0); // Error
```

Visual Representation

Producer

List<? extends Animal>

Dog

Dog

Dog

Read

↓

Animal

Cannot safely add.

Consumer

List<? super Dog>

Animal

Animal

Animal

Add Dog
Add Puppy

Cannot safely read as Dog.

Why "extends" is Read Only

Suppose

List<? extends Animal> list;

Actual object may be

List<Dog>

or

List<Cat>

Java doesn't know.

If

list.add(new Dog());

and actual list is

List<Cat>

Disaster.

Therefore Java forbids adding.

Why "super" is Write Only

Suppose

List<? super Dog> list;

Actual list could be

List<Object>

or

List<Animal>

Reading

```
Dog d = list.get(0);
```

Not safe.

Because list may contain

Animal

not Dog.

Hence Java returns only

Object

Real-world Example

Imagine a **Dog Shelter**.

Producer

Shelter gives Dogs

You receive Dogs

You can safely treat them as Animals.

Read only

Consumer

Imagine a **Dog Adoption Center**.

It accepts Dogs.

You can insert

Dog

Puppy

because both are Dogs.

Reading?

Not guaranteed.

Could contain

Animal
Object

Java Library Example

Collections.copy()

Signature

```
public static <T> void copy(  
    List<? super T> dest,  
    List<? extends T> src)
```

Notice both PECS rules.

Source

Producer

extends

Destination

Consumer

super

Example

```
List<Dog> source = List.of(  
    new Dog(),  
    new Dog()  
);
```

```
List<Animal> destination = new ArrayList<>();
```

```
Collections.copy(destination, source);
```

Source produces Dogs.

Destination consumes Dogs.

Another Example

```
public static void printAnimals(  
    List<? extends Animal> list)  
{  
    for (Animal a : list)  
        System.out.println(a);  
}
```

Call

```
printAnimals(dogs);
```

```
printAnimals(cats);
```

```
printAnimals(animals);
```

Works for every subclass.

Another Consumer Example

```
public static void addDogs(  
    List<? super Dog> list)  
{  
    list.add(new Dog());  
  
    list.add(new Puppy());  
}
```

Call

```
addDogs(new ArrayList<Animal>());
```

```
addDogs(new ArrayList<Object>());
```

```
addDogs(new ArrayList<Dog>());
```

All valid.

Interview Question

Difference between extends and super

| Feature | <? extends T> | <? super T> |
|-------------|---------------------------|----------------------------------|
| Meaning | T or subclass | T or superclass |
| Reads | Safe as T | Only safe as Object |
| Writes | Not allowed (except null) | Allowed for T and its subclasses |
| Role | Producer | Consumer |
| Flexibility | Read-only | Write-focused |

Decision Algorithm

When writing a generic method, ask:

1. **Am I only reading elements from the collection?**
 - Use <? extends T>.
2. **Am I only adding elements to the collection?**
 - Use <? super T>.
3. **Am I both reading and writing?**
 - Use a concrete type such as List<T> instead of a wildcard.

Summary

- **PECS** means **P**roducer **E**xtends, **C**onsumer **S**uper.
- Use <? **extends** T> when the collection is a **producer** of data. You can safely read elements as T, but you cannot add new elements (except null).
- Use <? **super** T> when the collection is a **consumer** of data. You can safely add T objects and their subclasses, but when reading, the returned type is only guaranteed to be Object.
- If a method needs to both read and write elements of the same type, prefer **List<T>** rather than wildcard bounds.

Remember this interview shortcut:

"Read = Extends, Write = Super."